

Task:3 Array Manipulation and Analysis programs

(A) Return the max sliding window

Aim:-

To write a java program to find the maximum value in each sliding window of size k as the window moves from left to right across a given integer array.

Algorithm:-

1. Start
2. Read the array nums and window size k
3. Create an output array of size $n-k+1$
4. For each window starting from index $i=0$ to $n-k$:
 - * Assume the first element of the window is the maximum
 - * Compare all k elements in the current window
 - * Update the maximum value
 - * Store the maximum value in the result array
5. Print the result.

Program:-

```
import java.util.Scanner;

class sliding window Maximum{
    public sliding window void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print ("Enter array size:");
        int n = sc.nextInt ();
        int [] num = new int [n];
        System.out.println ("Enter " + n + " array elements:");
        for (int i=0; i<n; i++) {
```

output:-

Enter array size : 8

Enter 8 array element : 1 3 -1 -3 5 3 6 7

1 3 -1 -3 5 3 6 7

Enter window size : 3

Sliding window maximum : 3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

3 3 5 5 6 7

```

num[i] = sc.nextInt(n-k+1);
for (int i=0; i<n-k; i++)
    nums[i] = sc.nextInt();
}
System.out.print("Enter window size:");
int k = sc.nextInt();
int[] result = new int[n-k+1];
for (int i=0; i<n-k; i++){
    int max = nums[i];
    for (int j=i; j<i+k; j++){
        if (nums[j] > max) {
            max = nums[j];
        }
        result[i] = max;
    }
}
System.out.print("sliding window maximum\n");
for (int i=0; i<result.length; i++){
    System.out.print(result[i] + " ");
}
}
}

```

Result:-

The Java program successfully finds the maximum element in each sliding window of the size k. The output correctly represents the maximum values for all window positions moves from left to right

(B) ADJUSTED SUM:-

Aims- To write a Java program that calculates the adjusted sum of an integer array by adding all even numbers and subtracting all odd numbers.

Algorithms-

1. Start the program
2. Read the value of N (size of the array)
3. Read N integer elements into the array.
4. Initialize a variable sum to 0
5. Traverse each element of the array.
 - * If the element is even, add it to sum.
 - * If the element is odd, subtract it from sum.
6. Display the final adjusted sum.
7. Stop the program.

Program:-

```
import java.util.Scanner;  
class adjusted sum {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter the size of array: ");  
        int N = sc.nextInt();  
        int sum = 0;  
        System.out.print("\nEnter the array elements:");  
        int N = sc.nextInt();  
        int[] arr = new int[N];
```

Output:-

Enter the size of the array: 5

Enter the array elements:

10 15 20 25 30

Adjusted sum 20

```
int arr[5], sum = 0, i;
for(i = 0; i < 5; i++)
    arr[i] = scanf("%d", &arr[i]);
sum = arr[0];
```

```
for(i = 1; i < 5; i++)
    sum = sum + arr[i];
printf("Sum of array is: %d", sum);
```

```
sum = arr[0];
for(i = 1; i < 5; i++)
    sum = sum + arr[i];
```

```
printf("Sum of array is: %d", sum);
```

```
int sum = 0;
system.out.println("Enter the array elements");
for (int i = 0; i < N; i++) {
    arr[i] = sc.nextInt();
    if (arr[i] % 2 == 0) {
        sum = sum + arr[i];
    } else {
        sum = sum - arr[i];
    }
}
system.out.println("Adjusted sum = " + sum);
}
```

Result:-

Thus, the Java program calculates the adjusted sum from the array by adding even element and subtracting odd element.

(C) Rotate Array and Find Max difference

Aim:-

To write a java program that rotates an array to the right by K positions and finds that maximum absolute difference between two adjacent elements after rotation.

Algorithm:-

1. start the program.
2. Read the size of array N.
3. Read N elements into the array.
4. Read the value of K.
5. Rotate the array to the right by K positions using the formula $rotated[(i+K)\%N] = arr[i]$
6. Traverse the rotated array and calculate the absolute difference b/w adjacent elements using $math.abs()$
7. store the maximum difference found.
8. Display the rotated array and the max difference.
9. Stop the program.

Program

```
import java.util.Scanner;  
class Rotate Max Difference {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter the size of the array:");  
        int N = sc.nextInt();  
        int[] arr = new int[N];  
        int[] rotated = new int[N];
```

Output:-

Enter the size of the array: 5

Enter the array element

3 8 9 7 6

Enter K value: 7

Rotated Array:

6 3 8 9 7

Maximum Difference = 5.

```

system.out.println("Enter the array elements:");
for (int i=0; i<N; i++) {
    arr[i] = sc.nextInt();
}

```

```

system.out.print("Enter k value:");
int k = sc.nextInt();
k = k % N; // Handle cases where k > N

```

```

// Rotate array to the right
for (int i=0; i<N; i++) {
    rotated[(i+k)%N] = arr[i];
}

```

```

// Find the maximum absolute difference
int maxDiff = 0;
for (int i=0; i<N-1; i++) {
    int diff = Math.abs(rotated[i] - rotated[i+1]);
    if (diff > maxDiff) {
        maxDiff = diff;
    }
}

```

```

// Display rotated array
system.out.println("Rotated Array:");
for (int num: rotated) {
    system.out.print(num + " ");
}
system.out.println("Maximum diff" + maxDiff);

```

TECH - USE	
EX NO.	
PERFORMANCE	
RESULT AND ANALYSIS (5)	
VIVA VOCE (5)	
RECORD (5)	
TOTAL (20)	
SIGN WITH DATE	

Results

Thus the program successfully rotates the array to the right by k positions and find the maximum absolute difference b/w adjacent elements.